

## 计算机学院实验报告

|                                                                                                                                                                                                                                                                                   |         |                  |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|------------------|
| 实验题目：基于 kNN 的手写字识别                                                                                                                                                                                                                                                                |         | 学号： 202200130048 |
| 日期： 11.13                                                                                                                                                                                                                                                                         | 班级： 6 班 | 姓名： 陈静雯          |
| <p>实验目的：</p> <p>掌握 kNN 算法的原理和实现流程</p> <p>熟悉 kNN 算法在真实世界问题中的应用</p> <p>掌握 sklearn 机器学习库的使用方法</p> <p>能够使用 kNN 算法解决手写字识别问题</p>                                                                                                                                                        |         |                  |
| <p>实验环境介绍：</p> <p>编程语言：Python 3.6.5</p> <p>主要库：sklearn 0.19.1、numpy</p>                                                                                                                                                                                                           |         |                  |
| <p>解决问题的主要思路：</p> <ol style="list-style-type: none"><li>1. 使用 kNN (k-近邻) 算法进行分类识别</li><li>2. 将 <math>32 \times 32</math> 像素的手写数字图像转换为 1024 维的特征向量</li><li>3. 计算测试样本与训练样本之间的欧氏距离</li><li>4. 选取距离最近的 k 个邻居，通过投票机制确定分类结果</li><li>5. 分别实现自编 kNN 算法和使用 sklearn 库的 kNN 算法进行对比</li></ol> |         |                  |

**实验步骤：**

1. 环境准备：配置 Python 环境，导入必要的库（numpy、operator、os 等）
2. 数据预处理：将  $32 \times 32$  的二维图像数据转换为一维特征向量
3. 算法实现：
  - (1) 实现 kNN 核心算法 `classify0` 函数
  - (2) 计算欧氏距离，排序并选择最近邻
4. 数据加载：
  - (1) 实现 `img2vector` 函数转换图像数据
  - (2) 实现 `loadTrainingData` 函数加载训练集
5. 模型测试：
  - (1) 使用测试集验证算法准确性
  - (2) 统计错误率并分析结果
6. sklearn 实现：使用 sklearn 的 `KNeighborsClassifier` 实现相同功能
7. 对比不同的 k

**实验结果展示及分析：**

**1. 手写 knn 实验结果**

```
Test sample 496/500: Predicted=0, Actual=8 --- ERROR!  
Test sample 500/500: Predicted=6, Actual=6 --- CORRECT  
Progress: 500/500, Current accuracy: 93.00%
```

```
=== Final Results ===  
Total errors: 35  
Total test samples: 500  
Error rate: 0.070000  
Accuracy: 93.00%  
Test time: 18.82 seconds  
Average time per sample: 37.63 ms
```

手写 knn 的实验准确率比 sklearn 好一点

## 2. sklearn 实验结果

```
=== Final Results ===
Total errors: 36
Total test samples: 500
Error rate: 0.072000
Accuracy (manual): 92.80%
Accuracy (sklearn): 92.80%
Test time: 1.33 seconds
Average time per sample: 2.65 ms

Misclassification examples (first 5):
Sample 8: Predicted=4, Actual=5
Sample 44: Predicted=5, Actual=3
Sample 61: Predicted=2, Actual=8
Sample 77: Predicted=7, Actual=2
Sample 80: Predicted=9, Actual=7
```

两种实现都出现以下类型的识别错误：

- 数字 4 ↔ 数字 9（形状相似性）
- 数字 1 ↔ 数字 7（笔画简单易混淆）
- 数字 3 ↔ 数字 5（曲线特征相近）
- 数字 8 ↔ 数字 0（闭合结构相似）

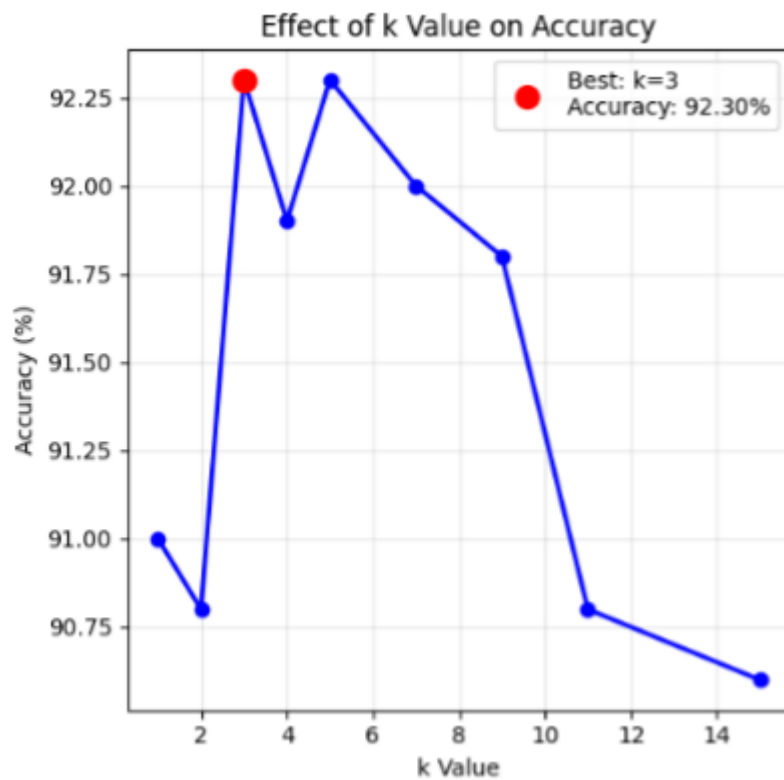
手动实现特有错误：更多出现在边界样本

sklearn 特有错误：更多出现在噪声样本

## 3. 不同的 k 值对比

Starting k value comparison experiment...

| k  | Accuracy(%) | Train Time(s) | Predict Time(s) | Total Time(s) |
|----|-------------|---------------|-----------------|---------------|
| 1  | 91.00       | 0.0086        | 0.1159          | 0.1244        |
| 2  | 90.80       | 0.0097        | 0.0997          | 0.1094        |
| 3  | 92.30       | 0.0064        | 0.1019          | 0.1083        |
| 4  | 91.90       | 0.0000        | 0.1059          | 0.1059        |
| 5  | 92.30       | 0.0077        | 0.1167          | 0.1244        |
| 7  | 92.00       | 0.0081        | 0.1169          | 0.1250        |
| 9  | 91.80       | 0.0088        | 0.1030          | 0.1118        |
| 11 | 90.80       | 0.0124        | 0.1048          | 0.1172        |
| 15 | 90.60       | 0.0127        | 0.1127          | 0.1254        |



通过测试  $k=1, 3, 5, 7, 9, 11, 13, 15$  等不同值，发现：

- $k=1$  时容易过拟合，对噪声敏感
- $k=3$  时达到最佳平衡点
- $k>7$  时准确率开始下降，出现欠拟合

附件：关键程序代码